

Popup + Behavioral AI Platform — Build Roadmap

Phase 0: Foundation & Scope Lock (before writing code)

- ☐ Define MVP scope in writing — what ships in v1 vs later (resist scope creep into the AI agent too early)
 - ☐ Pick legal structure for data handling: draft a basic DPA (Data Processing Agreement) template for customers
 - ☐ Decide target regions for launch (affects GDPR/CCPA requirements from day one)
 - ☐ Set up core infra accounts: hosting (Vercel/AWS), Postgres provider (Supabase/Neon/RDS), repo, CI/CD
-

Phase 1: Core Sellable Product (No AI Yet)

Goal: something you can sell to a real customer

- ☐ Next.js app scaffolded (App Router, TypeScript)
- ☐ Auth for customers (Clerk or Auth.js)
- ☐ Postgres schema: accounts, users, popup configs, gift/reward rules
- ☐ Popup builder UI (drag-and-drop or template-based v1 — templates first, full builder later)
- ☐ Widget: lightweight embeddable JS (Preact-based), loads via `<script>` tag
- ☐ Widget renders popup + spin-the-wheel game + form (name/email/phone capture)
- ☐ Basic consent banner logic built into widget (gate tracking until accepted)
- ☐ Reward/gift delivery logic (coupon code generation, email delivery)
- ☐ Billing integration (Stripe — subscription tiers)
- ☐ Basic analytics dashboard (impressions, conversions, emails captured — simple counts, no AI)
- ☐ Deploy + onboard first pilot customer

Exit criteria: a real business can install your widget, run a popup campaign, and see basic results.

Phase 2: Behavioral Tracking Infrastructure

Goal: start collecting the data the AI will eventually use

- ☐ Extend widget to capture: clicks, scroll depth, time-on-page, navigation, exit intent, sampled mouse movement
- ☐ Client-side event batching (send every few seconds, not per-event)
- ☐ Ingestion endpoint (thin, fast, pushes to queue)

- ☐ Queue setup (Redpanda or Kafka — start with Redpanda, simpler ops)
- ☐ Stream consumer service: writes raw events + computes session-level aggregates
- ☐ ClickHouse instance for event/behavioral storage
- ☐ Data isolation model: customer_id partitioning across Postgres + ClickHouse
- ☐ Update consent flow to explicitly cover full-site behavioral tracking (not just popup interaction)
- ☐ Observability: basic logging/monitoring on ingestion pipeline (catch silent data loss early)

Exit criteria: every pilot customer's site traffic is being tracked, stored, and queryable.

Phase 3: Basic A/B Testing (Statistical, No AI)

Goal: prove the optimization loop end-to-end before adding ML complexity

- ☐ Variant system: multiple popup designs/copy per campaign
- ☐ Fixed-split traffic allocation (simple % split across variants)
- ☐ Statistical significance calculation (basic frequentist test — sample size, conversion rate, confidence interval)
- ☐ Dashboard view: variant performance comparison
- ☐ Manual "declare winner" flow for customer

Exit criteria: customers can run and read results of a real A/B test without any AI involved.

Phase 4: Bandit-Based Auto-Optimization

Goal: real differentiation — adaptive optimization within a single customer's data

- ☐ Python service (FastAPI) for optimization logic, separate from Next.js
- ☐ Implement Thompson Sampling (or contextual bandit if factoring in device/traffic source/time)
- ☐ Redis for fast bandit state read/write (arm statistics)
- ☐ Internal API: Next.js ↔ Python bandit service
- ☐ Auto-reallocation of traffic toward winning variants (replacing fixed-split)
- ☐ Dashboard update: show live bandit allocation, not just static test results
- ☐ Guardrails: minimum exploration rate, minimum sample size before trusting results

Exit criteria: popups optimize themselves in near real-time per customer, without a human declaring a winner.

Phase 5: Industry Data Corpus & Cross-Customer Priors

Goal: solve the cold-start problem for low-traffic customers

- ☐ Build scraping pipeline for popular/high-performing popup designs (check ToS/legal boundaries)
- ☐ Cluster scraped data by industry/vertical
- ☐ Vector DB setup (pgvector or Qdrant) for similarity search over the corpus
- ☐ Aggregate anonymized performance patterns by industry from your own customer base
- ☐ Bandit prior initialization: new customers start from industry-informed priors instead of zero data
- ☐ Legal review: ensure cross-customer aggregation is anonymized/compliant, not raw PII sharing

Exit criteria: a brand-new low-traffic customer's popups perform reasonably well from day one, not after weeks of blind testing.

Phase 6: LLM Generative Agent Layer

Goal: the ambitious version — AI that creates and explains, not just allocates traffic

- ☐ RAG setup: LLM retrieves relevant examples from industry corpus via vector search
- ☐ Variant generation: LLM proposes new copy/design/offer variants for the bandit to test
- ☐ Plain-English insight generation: LLM explains why a variant won/lost to the customer
- ☐ Guardrails on LLM output (brand safety, no fabricated stats, consistent tone with customer's brand)
- ☐ Feedback loop: bandit results feed back into future LLM generation prompts
- ☐ Customer-facing "agent" UI (chat or report format for insights)

Exit criteria: the full loop is closed — agent generates variants, bandit tests them, agent explains results, cycle repeats autonomously.

Ongoing / Cross-Cutting (applies across all phases)

- ☐ Multi-tenant security review at each phase (data isolation between customers)
 - ☐ Consent/privacy compliance review at each phase (GDPR/CCPA as you add tracking depth)
 - ☐ Cost monitoring (ClickHouse storage, LLM API costs scale with usage — watch this from Phase 2 onward)
 - ☐ Customer feedback loop — talk to pilot customers before building Phase 4+ (validate they even want auto-optimization, or just want good reporting)
-

Suggested sequencing note

Phases 1-3 are enough to have a real, sellable product and validate demand before you invest in the harder ML/agent work (Phases 4-6). Don't skip straight to the agent — it's the most expensive and riskiest part to build, and it's much easier to build well once you have real customer data flowing through Phases 1-3.